

Processos e comunicação entre processos

fork, memória independente e pipe na prática

Apostila da aula de 5 de outubro de 2026

Weslei Ferreira Santos

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Material-base: Prof. Maycon Leone M. Peixoto. Os programas citados estão no repositório da disciplina e podem ser compilados com `make`.

Sumário

1	Antes de começar	3
1.1	O que você deve conseguir ao final	3
1.2	Como usar os exemplos	3
2	Programa e processo	4
2.1	Duas palavras que parecem sinônimas	4
2.2	O que o SO guarda sobre cada processo	4
2.3	O espaço de endereçamento	5
3	Criando processos com fork()	5
3.1	Uma chamada, dois retornos	5
3.2	O que o filho herda	6
3.3	getpid() e getppid()	6
3.4	Um exemplo completo	7
3.5	Quem executa primeiro?	7
3.6	Um detalhe de C: buffers de saída	8
4	fork() dentro de um laço	8
4.1	O programa fork2.c	8
4.2	Quantos processos existem?	10
5	Cada processo tem a sua memória	11
5.1	A previsão	11
5.2	Por que zero	11
5.3	Mesmo endereço, valores diferentes	12
5.4	E se o pai esperar os filhos?	12
6	Término e coleta: exit e waitpid	13
6.1	Como um processo termina	13
6.2	waitpid	13
6.3	Zumbis e órfãos	13
6.4	Esperar não é comunicar	14
7	Comunicação entre processos com pipe	14

7.1	IPC	14
7.2	O que é um pipe	14
7.3	Descritores de arquivo	15
7.4	A ordem das operações	15
7.5	read e write	16
7.6	Bloqueio e fim de arquivo	16
7.7	O exemplo pipe_soma.c	17
7.7.1	O helper transferir	18
7.7.2	A verificação final do pai	18
8	Vários processos: pipe2.c	18
8.1	O problema	18
8.2	Quem cria quem	19
8.3	Por onde os dados passam	19
8.4	Cada processo fica só com as pontas que usa	19
8.5	Os trechos principais	20
8.6	Execução	21
9	Atividade prática	21
9.1	O que fazer	21
9.2	Compilar e testar	21
9.3	Lista de conferência	22
9.4	Perguntas para responder em dupla	22
9.5	Erros comuns	22
10	Resumo	23
11	Exercícios de fixação	23
	Para saber mais	24

1 Antes de começar

Esta apostila acompanha a aula sobre processos e comunicação entre processos. Os slides mostram o caminho em poucas palavras; aqui cada ideia aparece com mais calma, com os programas completos, as saídas esperadas e os detalhes que costumam gerar dúvida.

A aula inteira gira em torno de uma pergunta:

Como um processo recebe o resultado calculado por outro?

Para respondê-la, seguimos uma ordem deliberada. Primeiro entendemos o que é um processo e como ele nasce com `fork()`. Depois constatamos, com um experimento, que pai e filho *não* compartilham a memória. Só então faz sentido apresentar o pipe, o mecanismo que o sistema operacional oferece para transportar dados de um processo a outro.

1.1 O que você deve conseguir ao final

- ▶ Identificar os valores que `fork()` devolve ao pai e ao filho e explicar de onde cada um continua a execução.
- ▶ Explicar por que uma variável alterada no filho não muda no pai.
- ▶ Desenhar a árvore de processos criada por um laço que chama `fork()`.
- ▶ Diferenciar *esperar o término* de um filho (`waitpid`) de *receber dados* dele (pipe).
- ▶ Completar um programa em que o filho envia um inteiro ao pai por um pipe.

1.2 Como usar os exemplos

Todos os programas citados ficam no repositório da disciplina. Em um terminal Linux, na raiz do repositório:

```
$ make test           # compila os exemplos e confere o comportamento
$ make run-fork-basico # retornos de fork
$ make run-memoria    # mesmo endereco, valores diferentes
$ make run-fork       # fork2.c
$ make run-pipe-soma   # filho envia 30 ao pai
$ make run-pipe       # pipe2.c: raizes da equacao de 2o grau
```

Os exemplos são compilados com `-std=c11 -Wall -Wextra -Wpedantic -Werror`: qualquer aviso do compilador é tratado como erro. Rode cada programa enquanto lê a seção correspondente. Os números de PID das saídas mostradas aqui vão ser diferentes na sua máquina, e isso é esperado.

2 Programa e processo

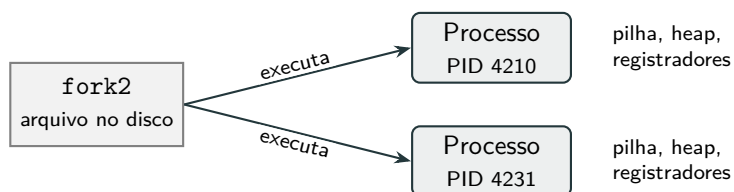
2.1 Duas palavras que parecem sinônimas

Um **programa** é um arquivo: uma sequência de instruções e dados guardada no disco. Ele não faz nada sozinho. Quando pedimos ao sistema operacional para executá-lo, o SO cria um **processo**.

Definição

Um **processo** é um programa em execução, junto com todo o estado necessário para executá-lo: o identificador (PID), o conteúdo dos registradores, a memória (código, dados, heap e pilha), os arquivos abertos e outras informações mantidas pelo núcleo do SO.

A diferença fica clara quando executamos o mesmo programa duas vezes, em dois terminais. Há um único arquivo e dois processos. Cada um tem o seu PID, a sua pilha, o seu heap e os seus registradores.



Os PIDs da figura são ilustrativos.

2.2 O que o SO guarda sobre cada processo

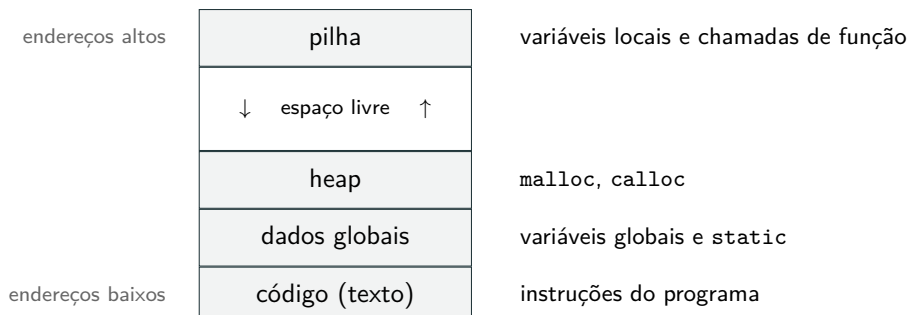
Para cada processo, o núcleo mantém um registro (frequentemente chamado de *bloco de controle do processo*, ou PCB). Entre as informações guardadas estão:

- ▶ **PID**: o número que identifica o processo enquanto ele existe.
- ▶ **PPID**: o PID do processo que o criou (o pai).
- ▶ **Estado**: executando, pronto, bloqueado esperando algo, terminado.
- ▶ **Contexto**: o valor dos registradores, incluindo o contador de programa, salvo quando o processo sai da CPU.
- ▶ **Espaço de endereçamento**: a memória que o processo enxerga.
- ▶ **Tabela de descritores**: os arquivos, pipes e terminais que o processo tem abertos.

No Linux, essas informações aparecem em comandos como `ps -f` e `pstree -p`, vistos na aula de introdução ao Linux.

2.3 O espaço de endereçamento

Cada processo enxerga a sua memória como se fosse a única do computador. Essa memória está organizada em regiões:



Os endereços que o programa usa são **virtuais**: o hardware e o SO traduzem cada endereço virtual para uma posição física da memória, e essa tradução é diferente para cada processo. Esse detalhe vai explicar, na Seção 5, por que o mesmo endereço pode guardar valores diferentes em dois processos.

Pare e pense

Se uma execução de um programa altera uma variável, a outra execução, em outro terminal, necessariamente observa a alteração? Anote sua resposta. Vamos testá-la nas próximas seções.

3 Criando processos com fork()

3.1 Uma chamada, dois retornos

No Linux e em outros sistemas Unix, um processo cria outro com a chamada de sistema `fork()`. Ela duplica o processo que a chamou: o original passa a ser o **pai** e a cópia é o **filho**.

```
#include <unistd.h>

pid_t fork(void);
```

O aspecto mais estranho de `fork()` é que ela é chamada uma vez e **retorna duas vezes**: uma no pai e outra no filho. O valor de retorno é o que permite a cada processo saber quem ele é.

Onde	Valor devolvido	Significado
No filho	0	"Eu sou o filho."
No pai	PID do filho (positivo)	"Criei este filho."
No pai, em caso de falha	-1	Nenhum filho foi criado; errno diz o motivo.

Por isso o código que usa `fork()` quase sempre tem a mesma forma:

```
pid_t pid = fork();
if (pid == -1) {
    /* falha: nenhum filho foi criado nesta chamada */
} else if (pid == 0) {
    /* este caminho executa no filho */
} else {
    /* este caminho executa no pai; pid identifica o filho */
}
/* pai e filho podem chegar aqui */
```

O filho não recomeça o main

O filho é uma cópia do pai *no ponto da chamada*. Ele não executa as linhas anteriores a `fork()`: continua logo depois dela, exatamente como o pai, só que recebendo 0 como retorno. Variáveis criadas antes da chamada já existem no filho, com os mesmos valores que tinham no pai naquele instante.

3.2 O que o filho herda

Logo após um `fork()` bem-sucedido, o filho tem:

- ▶ uma **cópia** de toda a memória do pai: código, variáveis globais, heap e pilha;
- ▶ os mesmos valores nos registradores, inclusive o ponto do programa onde a execução continua;
- ▶ cópias dos **descritores de arquivo** abertos, que apontam para os mesmos arquivos e pipes que os do pai (isso será essencial na Seção 7);
- ▶ um PID **novo** e, como PPID, o PID do pai.

Copiar toda a memória a cada `fork()` seria caro. O Linux usa a técnica de **cópia na escrita** (*copy-on-write*): logo após a chamada, pai e filho apontam para as mesmas páginas físicas, marcadas como somente leitura. Quando um dos dois tenta escrever numa página, o SO faz uma cópia dela só para quem escreveu. Para o programador o efeito é o mesmo de uma cópia completa: **o que um escreve, o outro não vê**.

3.3 `getpid()` e `getppid()`

Duas chamadas simples ajudam a acompanhar quem é quem:

- ▶ `getpid()` devolve o PID do processo que está executando a chamada;
- ▶ `getppid()` devolve o PID do pai desse processo.

Note a diferença: no pai, o valor devolvido por `fork()` é o PID do *filho*, e não o do próprio pai.

3.4 Um exemplo completo

O programa `examples/dia05/fork_basico.c` mostra os dois retornos lado a lado:

```
1  /* Aula de 5/10: os dois retornos de fork e a coleta do filho. */
2  #include <errno.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <sys/wait.h>
6  #include <unistd.h>
7
8  int main(void)
9  {
10     printf("Antes do fork: PID %d\n", (int)getpid());
11     fflush(stdout); /* Evita que o filho herde texto ainda no buffer. */
12
13     pid_t pid = fork();
14     if (pid == -1) { perror("fork"); return EXIT_FAILURE; }
15     if (pid == 0) {
16         printf("Filho: fork retornou 0; meu PID %d; meu pai %d\n",
17               (int)getpid(), (int)getppid());
18         fflush(stdout); /* _exit nao esvazia buffers de stdio. */
19         _exit(EXIT_SUCCESS);
20     }
21     printf("Pai: fork retornou %d; meu PID %d\n", (int)pid, (int)getpid());
22
23     int status;
24     pid_t coletado;
25     do { coletado = waitpid(pid, &status, 0); } while (coletado == -1 && errno == EINTR);
26     if (coletado == -1) { perror("waitpid"); return EXIT_FAILURE; }
27     if (WIFEXITED(status))
28         printf("Pai: filho %d terminou com codigo %d\n", (int)coletado, WEXITSTATUS(status));
29     ;
30     return EXIT_SUCCESS;
31 }
```

Uma execução possível:

```
$ make run-fork-basico
Antes do fork: PID 2020435
Filho: fork retornou 0; meu PID 2020436; meu pai 2020435
Pai: fork retornou 2020436; meu PID 2020435
Pai: filho 2020436 terminou com codigo 0
```

Repare que o número devolvido ao pai (2020436) é o PID que o filho obtém com `getpid()`, e que o `getppid()` do filho é o PID do pai.

3.5 Quem executa primeiro?

Depois de `fork()`, pai e filho são dois processos independentes, disputando a CPU. O escalonador do SO decide quem roda, em que ordem e por quanto tempo. Em computadores com vários núcleos, os dois podem até rodar ao mesmo tempo.

Não conte com a ordem

Na saída acima, a linha do filho apareceu antes da linha do pai. Em outra execução a ordem pode se inverter. Um programa correto não pode depender disso: se a ordem importa, ela precisa ser imposta com sincronização, como `waitpid` ou um pipe.

3.6 Um detalhe de C: buffers de saída

As funções de `stdio.h`, como `printf`, não escrevem diretamente no terminal. Elas acumulam o texto num *buffer* na memória do processo e só o entregam ao SO quando o buffer enche, quando aparece uma quebra de linha (se a saída é um terminal) ou quando o programa termina normalmente.

Esse buffer faz parte da memória e, portanto, é copiado pelo `fork()`. Duas consequências:

1. Texto que ainda estava no buffer antes do `fork()` pode ser impresso **duas vezes**, uma por processo. É por isso que `fork_basico.c` chama `fflush(stdout)` antes de `fork()`.
2. A função `_exit` encerra o processo imediatamente, **sem** esvaziar esses buffers. Se o filho imprime algo e chama `_exit`, precisa chamar `fflush(stdout)` antes, ou o texto pode se perder. Isso acontece de fato quando a saída é redirecionada para um arquivo ou para outro programa.

Por que então o filho usa `_exit` e não `exit`? Porque `exit` esvaziaria também os buffers *herdados* do pai e executaria funções registradas pelo pai com `atexit`. Em um filho, `_exit` encerra apenas o que é dele.

4 fork() dentro de um laço

4.1 O programa fork2.c

O primeiro exemplo da aula coloca `fork()` dentro de um laço de três iterações. Cada processo filho escreve um valor no vetor `v`; no final, apenas o processo original soma o vetor e imprime o resultado.

```
1  /* Aula de SO: todos os processos continuam o laço de fork. */
2  #include <errno.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <sys/wait.h>
6  #include <unistd.h>
7
8  int main(void)
9  {
10     const int tamanho = 3;
11     int *v = calloc((size_t)tamanho, sizeof *v);
12     pid_t original = getpid();
13     int falhou = 0;
14     if (v == NULL) { perror("calloc"); return EXIT_FAILURE; }
15
16     for (int i = 0; i < tamanho; ++i) {
17         pid_t pid = fork();
```

```
18     if (pid == -1) { perror("fork"); falhou = 1; break; }
19     if (pid == 0) v[i] = i + 5; /* Altera apenas a copia do filho. */
20 }
21 /* Cada processo coleta seus proprios filhos diretos. */
22 for (;;) {
23     int status;
24     pid_t pid = waitpid(-1, &status, 0);
25     if (pid > 0) {
26         if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) falhou = 1;
27     } else if (errno == EINTR) continue;
28     else if (errno == ECHILD) break;
29     else { perror("waitpid"); falhou = 1; break; }
30 }
31 if (getpid() == original) {
32     int soma = 0;
33     for (int i = 0; i < tamanho; ++i) soma += v[i];
34     printf("Soma total no pai: %d\n", soma);
35 }
36 free(v);
37 return falhou ? EXIT_FAILURE : EXIT_SUCCESS;
38 }
```

Alguns pontos do código antes da análise:

- ▶ `calloc` reserva o vetor **já zerado**. A versão original usava `malloc`, que não inicializa a memória; a soma lia valores indeterminados.
- ▶ `original` guarda o PID do processo que começou o programa. Todos os filhos herdam esse número, mas só um processo tem `getpid() == original`.
- ▶ Depois do laço, cada processo espera **os seus próprios filhos** com `waitpid(-1, ...)` até receber `ECHILD`, que significa “não há mais filhos para coletar”.

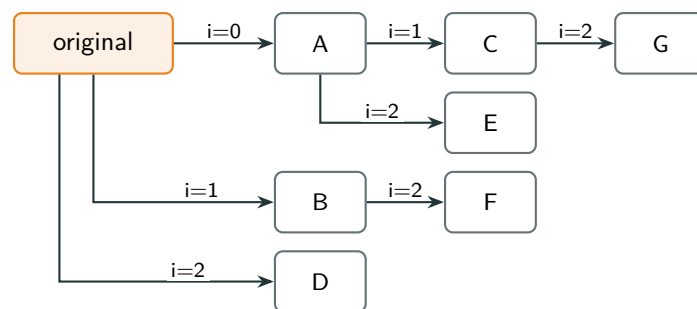
4.2 Quantos processos existem?

O ponto central é que **os filhos também continuam o laço**. Um filho criado na iteração $i = 0$ executa as iterações $i = 1$ e $i = 2$, criando os seus próprios filhos.

Vamos acompanhar iteração por iteração, supondo que todas as chamadas tenham sucesso. Chamaremos o processo inicial de *original* e os demais de A a G.

Iteração	Quem chama <code>fork()</code> e quem nasce	Processos no total
antes do laço	apenas o original	1
$i = 0$	original cria A	2
$i = 1$	original cria B; A cria C	4
$i = 2$	original cria D; A cria E; B cria F; C cria G	8

A árvore resultante, com o valor de i em que cada filho nasceu:



A cada iteração, o número de processos dobra. Com n iterações:

- ▶ existem 2^n processos ao todo, incluindo o original (aqui, $2^3 = 8$);
- ▶ $2^n - 1$ deles são novos (aqui, 7);
- ▶ foram executadas $1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1$ chamadas de `fork()`, embora o código tenha **uma só** chamada escrita.

Oito criados não é o mesmo que oito vivos

A contagem diz quantos processos foram criados ao longo da execução. Não significa que os oito estejam vivos ao mesmo tempo: dependendo do escalonamento, um processo pode terminar antes que outro seja criado.

Pare e pense

E se cada filho saísse do laço logo depois de nascer (por exemplo, com um `break` dentro do `if (pid == 0)`)? Nesse caso só o original completaria as três iterações, criando um filho em cada uma: seriam **4 processos** ao todo. Em geral, $n + 1$ em vez de 2^n .

5 Cada processo tem a sua memória

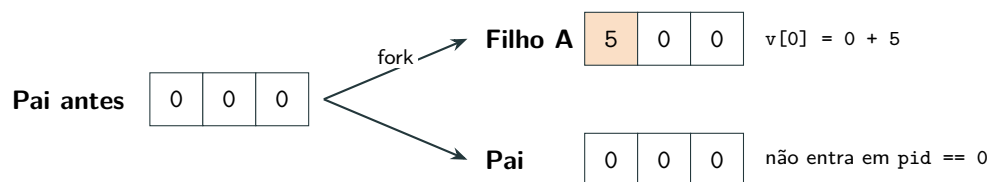
5.1 A previsão

Antes de executar `fork2.c`, vale fazer a previsão. Os filhos escrevem $v[0] = 5$, $v[1] = 6$ e $v[2] = 7$. Se essas escritas chegassem ao vetor do original, a soma seria $5 + 6 + 7 = 18$. O que o programa imprime?

```
$ make run-fork
./build/fork2
Soma total no pai: 0
```

5.2 Por que zero

O original nunca entra no ramo `if (pid == 0)`: para ele, `fork()` sempre devolve o PID de um filho. Portanto o original nunca escreve em `v`. Cada filho escreve na **sua cópia** do vetor, e essa escrita não alcança a memória de mais ninguém.



Fica mais interessante quando olhamos o vetor de cada um dos oito processos. Um filho herda o vetor *como estava no pai* no momento do `fork()`, e depois escreve a sua própria posição:

Processo	Criado por	Vetor ao final	Soma
original	(início)	[0, 0, 0]	0
A	original, $i = 0$	[5, 0, 0]	5
B	original, $i = 1$	[0, 6, 0]	6
D	original, $i = 2$	[0, 0, 7]	7
C	A, $i = 1$	[5, 6, 0]	11
E	A, $i = 2$	[5, 0, 7]	12
F	B, $i = 2$	[0, 6, 7]	13
G	C, $i = 2$	[5, 6, 7]	18

A soma 18 existe, mas só na memória de G, que herdou as escritas dos seus ancestrais A e C. Como G não é o original, ele não imprime nada. O original, que imprime, tem o vetor zerado.

5.3 Mesmo endereço, valores diferentes

Uma objeção comum: “mas o endereço de `v` é o mesmo no pai e no filho”. É verdade, e o programa `memoria_independente.c` mostra isso:

```
1  /* Aula de 5/10: o mesmo endereco virtual guarda valores diferentes. */
2  #include <errno.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <sys/wait.h>
6  #include <unistd.h>
7
8  int main(void)
9  {
10     int x = 10;
11     pid_t pid = fork();
12     if (pid == -1) { perror("fork"); return EXIT_FAILURE; }
13     if (pid == 0) {
14         x = 99; /* Altera somente a copia do filho. */
15         printf("Filho: x = %d em %p\n", x, (void *)&x);
16         fflush(stdout); /* _exit nao esvazia buffers de stdio. */
17         _exit(EXIT_SUCCESS);
18     }
19     int status;
20     pid_t coletado;
21     do { coletado = waitpid(pid, &status, 0); } while (coletado == -1 && errno == EINTR);
22     if (coletado == -1) { perror("waitpid"); return EXIT_FAILURE; }
23     printf("Pai:   x = %d em %p\n", x, (void *)&x);
24     return EXIT_SUCCESS;
25 }
```

```
$ make run-memoria
Filho: x = 99 em 0x7ffe72f6ea88
Pai:   x = 10 em 0x7ffe72f6ea88
```

O endereço impresso é o mesmo, e os valores são diferentes. Não há contradição: o endereço é **virtual**. O número `0x7ffe72f6ea88` é interpretado dentro do espaço de endereçamento de cada processo, e o SO o traduz para posições físicas diferentes no pai e no filho. Mesmo endereço virtual não significa mesma memória.

5.4 E se o pai esperar os filhos?

O `fork2.c` já espera todos os filhos antes de somar, e mesmo assim a soma é zero. Esperar não resolve, porque `waitpid` não foi feito para transportar dados. Vamos ver o que ele faz.

6 Término e coleta: exit e waitpid

6.1 Como um processo termina

Um processo termina quando retorna de `main`, quando chama `exit` ou `_exit`, ou quando recebe um sinal que o encerra (por exemplo, `SIGKILL`). Ao terminar, ele deixa um **código de saída**: por convenção, 0 (`EXIT_SUCCESS`) indica sucesso e qualquer outro valor indica falha.

6.2 waitpid

```
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *status, int opcoes);
```

- ▶ `pid` positivo espera aquele filho específico; `-1` espera qualquer filho.
- ▶ Em `status`, o SO informa como o filho terminou.
- ▶ Com `opcoes = 0`, a chamada **bloqueia** até algum filho terminar.
- ▶ Devolve o PID do filho coletado, ou `-1` em caso de erro.

O inteiro `status` é codificado e deve ser lido com macros:

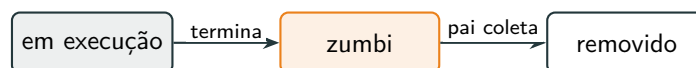
Macro	Pergunta que responde
<code>WIFEXITED(status)</code>	O filho terminou normalmente (por <code>exit</code> ou retorno de <code>main</code>)?
<code>WEXITSTATUS(status)</code>	Se sim, qual foi o código de saída?
<code>WIFSIGNALED(status)</code>	O filho foi encerrado por um sinal?
<code>WTERMSIG(status)</code>	Se sim, qual sinal?

Dois valores de `errno` aparecem nos exemplos:

- ▶ `EINTR`: a espera foi interrompida por um sinal antes de algum filho terminar. Basta chamar `waitpid` de novo, e é isso que o laço do `{ ... } while` dos exemplos faz.
- ▶ `ECHILD`: o processo não tem (mais) filhos para esperar. Em `fork2.c`, é a condição de saída do laço de coleta.

6.3 Zumbis e órfãos

Quando um filho termina, o SO não pode descartar tudo: o pai ainda pode querer saber o código de saída. O processo terminado fica então num estado especial, como **zumbi**, ocupando uma entrada na tabela de processos até o pai coletá-lo com `wait` ou `waitpid`. No ps, aparece com estado Z.



Se o pai termina antes do filho, o filho fica **órfão** e é adotado por outro processo (normalmente o `init/systemd`, PID 1), que se encarrega de coletá-lo. Um zumbi que dura muito tempo é sinal de um pai que esqueceu de chamar `waitpid`.

6.4 Esperar não é comunicar

O que <code>waitpid</code> faz	O que <code>waitpid</code> não faz
Espera um filho terminar.	Não copia a memória do filho para o pai.
Informa como ele terminou (código ou sinal).	Não cria memória compartilhada.
Libera a entrada do zumbi.	Não transporta resultados calculados.

Ideia-chave

`waitpid` coordena o **término**. O código de saída tem só 8 bits e serve para indicar sucesso ou falha, não para devolver dados. Para levar um resultado do filho ao pai, os bytes precisam ser enviados explicitamente por algum mecanismo do SO.

7 Comunicação entre processos com pipe

7.1 IPC

Como a memória de cada processo é isolada, o SO oferece mecanismos para que processos troquem dados. Eles recebem o nome geral de **IPC** (*Inter-Process Communication*). Pipes, sockets, filas de mensagens e memória compartilhada são exemplos. Nesta aula usamos o mais simples deles: o pipe.

7.2 O que é um pipe

Definição

Um **pipe** é um canal de bytes **unidirecional** mantido pelo núcleo do SO. Ele tem duas pontas: tudo o que é escrito numa ponta pode ser lido, na mesma ordem, na outra.

Você já usa pipes no terminal: em `ps aux | grep sleep`, o shell cria um pipe, liga a saída de `ps` à ponta de escrita e a entrada de `grep` à ponta de leitura.

```
#include <unistd.h>

int pipe(int fd[2]); /* 0 em sucesso, -1 em erro */
```

A chamada preenche o vetor com dois **descritores de arquivo**:

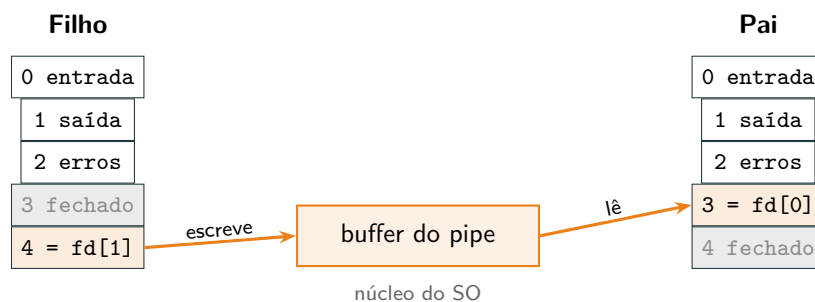
- ▶ `fd[0]`: a ponta de **leitura**;
- ▶ `fd[1]`: a ponta de **escrita**.

Uma forma de lembrar: 0 é a entrada padrão (lemos dela) e 1 é a saída padrão (escrevemos nela).

7.3 Descritores de arquivo

Um descritor é um número inteiro pequeno que o processo usa para se referir a um recurso aberto: um arquivo, um terminal, um pipe. Cada processo tem a sua **tabela de descritores**; o número é um índice nessa tabela. Os três primeiros já vêm abertos: 0 (entrada padrão), 1 (saída padrão) e 2 (saída de erros). Por isso `pipe()` costuma devolver 3 e 4.

Quando o processo chama `fork()`, o filho recebe uma **cópia da tabela**. As entradas copiadas apontam para os *mesmos* recursos do núcleo. É isso que permite a comunicação: se o pipe é criado **antes** do `fork()`, pai e filho passam a ter referências ao mesmo canal.



A figura mostra o estado depois que cada processo fechou a ponta que não usa: o filho fechou a leitura (3) e o pai fechou a escrita (4).

7.4 A ordem das operações

1. `pipe(fd)`: o pai cria o canal.
2. `fork()`: o filho herda os dois descritores.
3. Cada processo **fecha a ponta que não usa**: o filho, que só escreve, fecha `fd[0]`; o pai, que só lê, fecha `fd[1]`.
4. O filho escreve com `write(fd[1], ...)`; o pai lê com `read(fd[0], ...)`.
5. Cada um fecha a ponta que usou quando termina.
6. O pai coleta o filho com `waitpid`.

Pipe criado depois do fork não serve

Se `pipe()` for chamado depois de `fork()`, pai e filho criam *dois pipes diferentes*, cada um visível só para quem o criou. Não há canal em comum.

7.5 read e write

```
ssize_t write(int fd, const void *buf, size_t n);
ssize_t read(int fd, void *buf, size_t n);
```

As duas chamadas trabalham com **bytes**. `write` copia até `n` bytes a partir do endereço `buf` para o canal; `read` copia até `n` bytes do canal para o endereço `buf`. Ambas devolvem quantos bytes foram de fato transferidos, ou `-1` em caso de erro.

Para enviar um `int`, passamos o endereço da variável e o seu tamanho:

```
write(fd[1], &valor, sizeof valor); /* no filho */
read(fd[0], &valor, sizeof valor); /* no pai */
```

O ponteiro não é compartilhado

`&valor` no filho e `&valor` no pai são endereços de variáveis locais de cada processo. O SO **copia os bytes** da variável do filho para o buffer do pipe e, depois, do buffer para a variável do pai. As duas variáveis continuam separadas.

Enviar os bytes de um `int` (ou de um `double`, como em `pipe2.c`) funciona porque pai e filho rodam na mesma máquina e representam números da mesma forma. Entre computadores diferentes seria preciso combinar um formato, como texto.

7.6 Bloqueio e fim de arquivo

O comportamento de `read` depende do estado do pipe:

Situação	O que <code>read</code> faz
Há dados no pipe	Devolve os bytes disponíveis, até <code>n</code> . Pode devolver <i>menos</i> do que o pedido.
Pipe vazio e alguma ponta de escrita ainda aberta	Bloqueia : o processo dorme até chegarem dados.
Pipe vazio e todas as pontas de escrita fechadas	Devolve 0: fim de arquivo (EOF).

Do lado da escrita, também há regras:

- ▶ O buffer do pipe tem capacidade limitada (64 KiB por padrão no Linux). Se estiver cheio, `write` bloqueia até o leitor consumir dados.
- ▶ Escritas de até `PIPE_BUF` bytes (4096 no Linux) são **atômicas**: não se misturam com escritas de outros processos no mesmo pipe. Um `int` cabe com folga.
- ▶ Escrever num pipe sem nenhuma ponta de leitura aberta gera o sinal `SIGPIPE`, que por padrão encerra o processo. Se o sinal for ignorado, `write` devolve `-1` com `errno = EPIPE`. O `pipe2.c` ignora `SIGPIPE` justamente para poder tratar esse erro.

Por que fechar as pontas que não usamos?

Suponha que o pai esqueça de fechar a sua cópia de `fd[1]`. Se o filho terminar sem escrever, o pipe fica vazio, mas ainda existe uma ponta de escrita aberta: a do próprio pai. O `read` do pai então espera para sempre, porque o EOF só chega quando **todas** as pontas de escrita estão fechadas, inclusive as cópias herdadas.

7.7 O exemplo `pipe_soma.c`

O filho calcula $10 + 20$ e envia o resultado ao pai:

```

1  /* Aula de 5/10: um filho envia um inteiro ao pai pelo pipe. */
2  #include <errno.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <sys/wait.h>
6  #include <unistd.h>
7
8  /* modo=1 envia; modo=0 recebe. Trata transferências parciais e EINTR. */
9  static int transferir(int fd, void *dado, size_t tamanho, int modo)
10 {
11     unsigned char *p = dado;
12     while (tamanho > 0) {
13         ssize_t n = modo ? write(fd, p, tamanho) : read(fd, p, tamanho);
14         if (n < 0 && errno == EINTR) continue;
15         if (n < 0) { perror(modo ? "write" : "read"); return -1; }
16         if (n == 0) {
17             fprintf(stderr, "Transferencia incompleta: nenhum byte recebido/enviado.\n");
18             return -1;
19         }
20         p += n;
21         tamanho -= (size_t)n;
22     }
23     return 0;
24 }
25
26 int main(void)
27 {
28     int fd[2], valor;
29     if (pipe(fd) == -1) { perror("pipe"); return EXIT_FAILURE; }
30     pid_t pid = fork();
31     if (pid == -1) {
32         perror("fork"); close(fd[0]); close(fd[1]); return EXIT_FAILURE;
33     }
34     if (pid == 0) {
35         close(fd[0]);
36         valor = 10 + 20;
37         int resultado = transferir(fd[1], &valor, sizeof valor, 1);
38         close(fd[1]);
39         _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
40     }
41     close(fd[1]);
42     int resultado = transferir(fd[0], &valor, sizeof valor, 0);
43     close(fd[0]);
44     int status;
45     pid_t coletado;
46     do { coletado = waitpid(pid, &status, 0); } while (coletado == -1 && errno == EINTR);
47     if (coletado == -1) { perror("waitpid"); return EXIT_FAILURE; }
48     if (resultado != 0 || !WIFEXITED(status) || WEXITSTATUS(status) != 0)

```

```
49     return EXIT_FAILURE;
50     printf("Resultado recebido: %d\n", valor);
51     return EXIT_SUCCESS;
52 }
```

```
$ make run-pipe-soma
./build/pipe-soma
Resultado recebido: 30
```

O helper transferir. Como read e write podem transferir menos bytes do que o pedido, chamá-las uma única vez não garante que o int inteiro foi enviado ou recebido. A função transferir resolve isso com um laço:

- ▶ modo = 1 chama write; modo = 0 chama read.
- ▶ A cada volta, avança o ponteiro p e diminui tamanho pelo número de bytes transferidos, até não restar nada.
- ▶ Se a chamada foi interrompida por um sinal (EINTR), tenta de novo.
- ▶ Se read devolve 0 antes do fim, o escritor fechou o canal cedo demais; isso é tratado como erro.

Na atividade prática você vai *usar* esse helper, não reescrevê-lo. O foco é o protocolo: quem fecha qual ponta, quem envia e quem recebe.

A verificação final do pai. O pai só imprime o valor se três coisas deram certo: a recepção (resultado == 0), o término normal do filho (WIFEXITED) e o código de saída zero. Aqui waitpid e o pipe trabalham juntos: o pipe trouxe o dado, e waitpid confirmou que o filho terminou bem.

8 Vários processos: pipe2.c

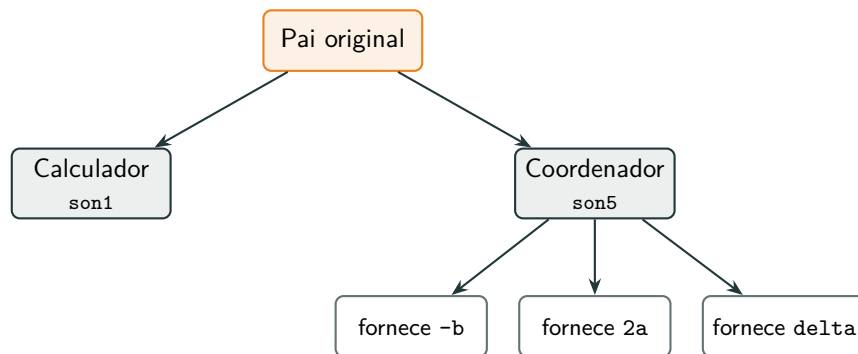
8.1 O problema

O pipe2.c calcula as raízes de uma equação do segundo grau usando seis processos e cinco pipes. O cálculo é pequeno e não fica mais rápido assim; o programa existe para mostrar vários canais funcionando ao mesmo tempo.

O usuário digita a, b e delta (já calculado; o programa não pede c). As raízes são

$$x_1 = \frac{-b - \sqrt{\Delta}}{2a}, \quad x_2 = \frac{-b + \sqrt{\Delta}}{2a}.$$

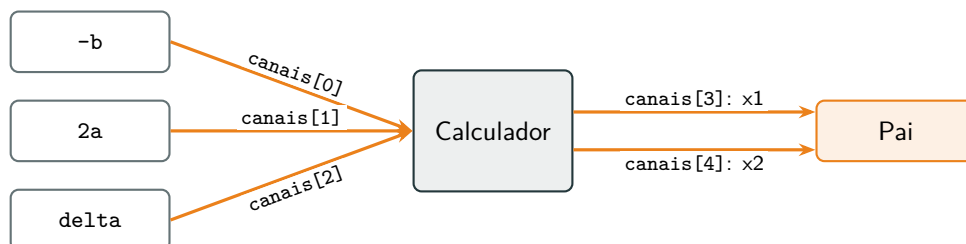
8.2 Quem cria quem



- ▶ O **pai** lê a entrada, cria os cinco pipes e depois os dois filhos diretos.
- ▶ O **calculador** recebe os três componentes, calcula as raízes e as envia ao pai.
- ▶ O **coordenador** cria três **fornecedores**, um para cada componente, e coleta esses filhos.

Os valores digitados são lidos pelo pai *antes* dos `fork()`, então todos os filhos já os herdam na memória. Mesmo assim, os fornecedores enviam os componentes ao calculador pelos pipes: o objetivo é exercitar o protocolo.

8.3 Por onde os dados passam



Árvore de criação e fluxo de dados são coisas diferentes

Os fornecedores são filhos do coordenador, mas não enviam nada a ele: enviam ao calculador, que é seu “tio”. Quem pode usar um pipe é quem tem um descritor dele, e todos herdaram os cinco canais do pai original. A árvore diz quem criou quem; os pipes dizem quem conversa com quem.

8.4 Cada processo fica só com as pontas que usa

A função `fechar_exceto` fecha todos os descritores dos cinco pipes, exceto os indicados. A tabela resume o que cada processo mantém aberto:

Processo	Lê de	Escreve em
Pai	canais[3], canais[4]	nenhum
Calculador	canais[0], canais[1], canais[2]	canais[3], canais[4]
Coordenador	nenhum	canais[0..2], só até criar os fornecedores
Fornecedor i	nenhum	canais[i]

O coordenador mantém as três pontas de escrita apenas para que os fornecedores as herdem; logo depois de criá-los, fecha todas. Se não fechasse, o calculador nunca veria EOF nesses canais caso um fornecedor falhasse.

8.5 Os trechos principais

O código completo está em `pipe2.c`. Os trechos abaixo concentram as ideias da aula.

```
74 static void calculador(void)
75 {
76     double menos_b, dois_a, delta, x1, x2;
77     fechar_exceto(0, 1, 2, 3, 4);
78     if (receber(canais[0][0], &menos_b) == -1 ||
79         receber(canais[1][0], &dois_a) == -1 ||
80         receber(canais[2][0], &delta) == -1) _exit(EXIT_FAILURE);
81     x1 = (menos_b - sqrt(delta)) / dois_a;
82     x2 = (menos_b + sqrt(delta)) / dois_a;
83     if (!isfinite(x1) || !isfinite(x2)) {
84         fprintf(stderr, "Raizes fora do intervalo numerico suportado.\n");
85         _exit(EXIT_FAILURE);
86     }
87     int falhou = enviar(canais[3][1], x1) == -1;
88     if (enviar(canais[4][1], x2) == -1) falhou = 1;
89     fechar_exceto(-1, -1, -1, -1, -1);
90     _exit(falhou ? EXIT_FAILURE : EXIT_SUCCESS);
91 }
```

O calculador fecha tudo o que não usa, recebe os três valores, calcula e envia as raízes. Os fornecedores são criados num laço dentro do coordenador:

```
93 static void fornecedores(double a, double b, double delta)
94 {
95     /* Coordenador conserva temporariamente as tres pontas de escrita. */
96     for (int i = 0; i < 5; ++i) {
97         close(canais[i][0]);
98         if (i >= 3) close(canais[i][1]);
99     }
100     double valores[3] = {-b, 2 * a, delta};
101     int falhou = 0;
102     for (int i = 0; i < 3; ++i) {
103         pid_t pid = fork();
104         if (pid == -1) { perror("fork fornecedor"); falhou = 1; break; }
105         if (pid == 0) {
106             for (int j = 0; j < 3; ++j)
107                 if (j != i) close(canais[j][1]);
108             int resultado = enviar(canais[i][1], valores[i]);
109             close(canais[i][1]);
110             _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
111         }
112     }
113     for (int i = 0; i < 3; ++i) close(canais[i][1]);
114     if (aguardar_filhos() == -1) falhou = 1;
115     _exit(falhou ? EXIT_FAILURE : EXIT_SUCCESS);
116 }
```

Note o `for` interno no filho: cada fornecedor fecha as pontas de escrita dos *outros* dois canais e fica só com a sua.

8.6 Execução

```
$ make run-pipe
Digite a, b e delta (delta ja calculado): 1 -3 1
Raizes: x1 = 1; x2 = 2
```

Conferindo: $-b = 3$, $2a = 2$ e $\sqrt{\Delta} = 1$. Então $x_1 = (3 - 1)/2 = 1$ e $x_2 = (3 + 1)/2 = 2$. Com a entrada 1 -2 0, o programa mostra uma raiz dupla ($x_1 = x_2 = 1$).

9 Atividade prática

9.1 O que fazer

Abra `examples/dia05/pipe_soma_atividade.c`. Ele é o `pipe_soma.c` com seis lacunas, marcadas em laranja abaixo:

```
if (pid == 0) {
    close(/* 1: ponta que o filho nao usa */);
    valor = /* 2: calculo */;
    int resultado = /* 3: enviar valor pelo helper transferir */;
    close(fd[1]);
    _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
close(/* 4: ponta que o pai nao usa */);
int resultado = /* 5: receber valor pelo helper transferir */;
close(/* 6: ponta usada pelo pai */);
```

Complete as lacunas para que:

- ▶ o filho feche a sua ponta de leitura, calcule $10 + 20$, envie o inteiro e feche a ponta de escrita;
- ▶ o pai feche a sua ponta de escrita, receba o inteiro, feche a ponta de leitura e imprima o valor.

Lembre a assinatura do helper: `transferir(descriptor, endereço do dado, quantidade de bytes, modo)`, com modo 1 para enviar e 0 para receber.

9.2 Compilar e testar

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    examples/dia05/pipe_soma_atividade.c -o /tmp/pipe-soma-aluno
$ /tmp/pipe-soma-aluno
Resultado recebido: 30
```

Antes de preencher, o esqueleto não compila. Isso é proposital.

9.3 Lista de conferência

- ▶ Cada processo fecha **exatamente uma** ponta antes de usar o canal, e é a ponta que ele não usa.
- ▶ O filho usa a ponta de escrita; o pai usa a de leitura.
- ▶ O tamanho passado ao helper é o tamanho do dado (`sizeof valor`), não o tamanho de um ponteiro.
- ▶ O endereço passado é o da variável (`&valor`), não o seu valor.
- ▶ Você consegue explicar por que escrever em `valor` no filho, sem o pipe, não mudaria `valor` no pai.

9.4 Perguntas para responder em dupla

1. Por que o pipe precisa ser criado antes do `fork()`?
2. Se o pai executar `read` antes de o filho escrever, isso é um erro? Quando o pai espera? Quando encontra EOF?
3. Por que os dois processos fecham pontas diferentes?

9.5 Erros comuns

Se você pensou...	Reveja...
"O pai soma 18."	Em qual processo cada atribuição acontece? Desenhe as memórias separadas.
"São quatro processos."	Siga um filho na próxima iteração: ele não sai do laço.
"waitpid compartilha memória."	Separe esperar o término de transportar os bytes.
"read sempre recebe tudo."	O retorno informa quantos bytes chegaram.
"Pipe vazio sempre dá EOF."	Alguma ponta de escrita ainda está aberta?
"Mesmo endereço, mesma memória."	O endereço é virtual e interpretado em cada processo.

10 Resumo

Chamada	Papel
<code>fork()</code>	Cria um processo filho com uma cópia independente da memória e dos descritores do pai. Devolve 0 ao filho, o PID do filho ao pai e -1 em caso de falha.
<code>getpid()</code> / <code>getppid()</code>	PID do próprio processo / PID do pai.
<code>_exit(codigo)</code>	Encerra o processo imediatamente, sem esvaziar buffers de stdio.
<code>waitpid(pid, &st, 0)</code>	Espera um filho terminar e coleta o seu estado. Coordena o término; não transporta dados.
<code>pipe(fd)</code>	Cria um canal unidirecional: <code>fd[0]</code> lê, <code>fd[1]</code> escreve.
<code>write</code> / <code>read</code>	Copiam bytes para o canal e do canal. Podem transferir menos que o pedido; <code>read</code> bloqueia com o canal vazio e devolve 0 no EOF.
<code>close(fd)</code>	Fecha um descritor. Fechar todas as pontas de escrita é o que gera EOF para o leitor.

Em três frases

`fork()` cria processos com memórias independentes. `waitpid` coordena o término e coleta os filhos. O pipe transporta os dados entre eles.

Próxima aula: 7/10

Threads de um mesmo processo **compartilham** a memória. Aí o problema se inverte: os dados chegam sem esforço, mas é preciso controlar quem acessa o quê e quando. Esse é o assunto de região crítica, Peterson e semáforos.

11 Exercícios de fixação

1. Em `fork2.c`, troque tamanho para 4. Quantos processos existem ao todo? Quantas chamadas de `fork()` são executadas?
2. Quantas vezes o programa abaixo imprime oi? Desenhe a árvore.

```
int main(void) { fork(); fork(); printf("oi\n"); return 0; }
```

3. O programa abaixo imprime AA no terminal, embora haja um único `printf`. Explique.

```
int main(void) { printf("A"); fork(); return 0; }
```

4. Na tabela da Seção 5, qual é o vetor do processo E e por quê?

5. Em `pipe_soma.c`, o que aconteceria se o pai não fechasse `fd[1]` e o filho terminasse com falha antes de escrever?
6. Para que pai e filho conversem nos dois sentidos (pergunta e resposta), quantos pipes são necessários? Por quê?
7. Por que o filho chama `_exit` em vez de `exit` ou `return`?
8. Por que `pipe2.c` ignora o sinal `SIGPIPE`?

Respostas

1. $2^4 = 16$ processos ao todo, 15 novos; também 15 chamadas executadas ($1 + 2 + 4 + 8$).
2. Quatro vezes. O primeiro `fork()` gera 2 processos; cada um executa o segundo `fork()`, gerando 4. Os quatro chegam ao `printf`.
3. O A fica no buffer de `stdout`, pois não há quebra de linha. O `fork()` copia o buffer. Cada processo, ao retornar de `main`, esvazia a sua cópia, e o A aparece duas vezes. Um `fflush(stdout)` antes do `fork()` evita a duplicação.
4. `[5, 0, 7]`. E foi criado por A na iteração `i = 2`. Herdou o vetor de A, que já tinha `v[0] = 5`, e escreveu `v[2] = 7`.
5. O `read` do pai bloquearia para sempre. O pipe estaria vazio, mas a ponta de escrita do próprio pai continuaria aberta, e o EOF nunca chegaria. O pai também nunca chegaria ao `waitpid`.
6. Dois. O pipe é unidirecional: um canal leva dados do pai ao filho e o outro, do filho ao pai. Usar um só canal nos dois sentidos faria um processo poder ler o que ele mesmo escreveu.
7. `exit` e `return` esvaziam os buffers de `stdio` herdados do pai (podendo duplicar saídas) e executam funções registradas pelo pai. `_exit` encerra só o filho, sem esses efeitos.
8. Se o leitor de um canal terminar antes, uma escrita nesse canal geraria `SIGPIPE` e encerraria o processo sem chance de tratamento. Ignorando o sinal, `write` devolve `-1` com `EPIPE`, e o programa trata a falha normalmente.

Para saber mais

- Páginas de manual do Linux: `man 2 fork`, `man 2 waitpid`, `man 2 pipe` e `man 7 pipe`. Também disponíveis em <https://man7.org/linux/man-pages/>.
- A. S. Tanenbaum e H. Bos. *Sistemas Operacionais Modernos*. Capítulo 2: processos e threads.
- A. Silberschatz, P. B. Galvin e G. Gagne. *Fundamentos de Sistemas Operacionais*. Capítulo 3: processos e comunicação entre processos.